

Föreläsning 11

Ändrad
26 Sep
12:38

Lade ni märke till att `isalnum()` (eng *is alpha-numeric*) är en kombination av `isalpha()` och `isdigit()`?

Vi kan implementera en egen version av `isalnum()` som fungerar likadant. Låt oss kalla den `IsAlnum()`.

```
1 #include <ctype.h>
2
3 int IsAlnum(char ch)
4 {
5     return isalpha(ch) || isdigit(ch);
6 }
```

Funktionen `IsAlnum()` returnerar *sant* om `isalpha(ch)` eller `isdigit(ch)` är sann. Jag utnyttjar den logiska operationen "eller". Den finns i C under namnet `||` -- två lodräta streck.

Med hjälp av `||` kan vi bygga upp logiska uttryck som kombinerar sanningsvärden från andra logiska uttryck. Förutom "eller" (`||`) finns operationerna "och" (`&&`) och "icke" (`!`).

I programspråket C är *sant*=1 och *falskt*=0. Logiska uttryck -- villkor -- är egentligen bara vanliga aritmetiska heltalsuttryck. Detta gäller även när vi sätter dem samman med de logiska operatorena.

Som sagt: "I C är allting heltal"!

p	q	!q	p && q	p q
0	0	1	0	0
0	1	0	0	1
1	0	1	0	1
1	1	0	1	1

Exempel:

```
■ while(i <= j && j <= k)...
■ if(isalpha(c) || c=='Å' || c=='Ä' ||
c=='Ö')...
■ while((c = getchar()) != EOF && c != 'q')...
■ k = (i == 2) || !(i < 5);
```

! Sant är inte bara 1, utan "allt som inte är noll", Så följande uttryck är också meningsfulla (men inte särskilt användbara):

```
i = !23; /* i = 0 */
i = 23 && 19; /* i = 1 */
```

Prioriteter

Om man blandar olika aritmetiska operatorer i samma uttryck gäller -- som ni vet från matematiken -- att `*` och `/` utförs före `+` och `-`. Man säger att `*` har *högre prioritet* än `+`.

De logiska operatorerna i C har lägre prioritet än jämförelseoperatorerna, som i sin tur har lägre prioritet än de aritmetiska operatorerna. Undantaget är `!` som binder starkare än de aritmetiska operatorerna.

Uttrycket

```
a * b * !c + d > e || f
```

beräknas därför i följande ordning:

```
(( (a * b) * (!c)) + d) > e || f
```

eller (oförutsägbart vilket):

```
((a * (b * (!c))) + d) > e || f
```

■ Operatorerna har följande prioritetsordning. Högsta prioritet står överst.

Prioritetsordning			
0	(x)		
1	x++	x--	
2	++x	--x	!x
3	x * y	x / y	x % y
4	x + y	x - y	
6	x < y	x > y	x <= y x >= y
7	x == y	x != y	
11	x && y		
12	x y		
13	x ? y : z		
14	x = y	x += y	x *= y etc...

Om du vill veta vad ett aritmetiskt uttryck betyder tar du fram tabellen ovan. Sedan sätter du parenteser runt deluttrycken. Börja med de högsta prioriteterna. Exempel:

```
a = b && c++ + d * e < f || g
a = b && (c++) + d * e < f || g
a = b && (c++) + (d * e) < f || g
a = b && ((c++) + (d * e)) < f || g
...
```

■ Genom att sätta ut parenteser kan programmeraren själv

bestämma beräkningsordningen. Med ett undantag:

! Kompilatorn får ignorera parenteserna i uttrycket $a + (b + c)$, och i stället göra $(a + b) + c$!
Denna frihet gäller dock endast $+$ och $*$ (och tre operatorer till som vi inte lärt oss ännu).

■ I uttrycken $(p \ || \ q)$ och $(p \ \&\& \ q)$ beräknas alltid p först. Om p är *sant* i $(p \ || \ q)$ beräknas inte q . Om p är *falskt* i $(p \ \&\& \ q)$ beräknas inte q .

Detta kan ha betydelse när programmeraren använder sideeffekter, dvs när villkoret innehåller tilldelning eller ändringsoperatorer:

```
if (i++ > 0 || --j < 0)
```

Endast om uttrycket $(i++ > 0)$ är *falskt*, kommer $(--j)$ att utföras.

Fetning igen

Programmet som fetade siffror från förra föreläsningen hade en slösaktig attityd: den ramade in varje siffra. Nu ska ni få se tre versioner som ramar in hela grupper av siffror istället.

Först en provkörning som gäller alla tre versionerna:

```
1 Bokstäver: abc. Siffror 123 456.
2 Bokstäver: abc. Siffror <b>123</b> <b>456</b>.
3 Decimaltal 12.345.
4 Decimaltal <b>12</b>.<b>345</b>.
5 Avslutande tal: -125
6 Avslutande tal: -<b>125</b>
7 ^d
```

Den första versionen av `PrintBoldDigits()` utnyttjar den logiska variabeln `ReadingDigits`. Den håller reda på när vi läser siffror.

```
1 /*
2 Exempel 16. PT Ding+Eing 1997
3 Jan Tångring, 1997-09-19
4
5 Rama in tal med paret <b>1234</b>
6
7 Nyheter:
8   - hängande else (dangling else)
9   - strängmakro
10  - putchar()
11 */
12
13 #include <stdio.h>
14 #include <ctype.h>
15
16 #define BOLD    "<b>"
17 #define NO_BOLD "</b>"
18
19 void PrintBoldDigits()
20 {
21     int ReadingDigits = 0;
22     int ch;
23
24     while ((ch = getchar()) != EOF) {
25         if (ReadingDigits) {
26             if (!isdigit(ch)) {
```

```

27         ReadingDigits = 0;
28         printf(NO_BOLD);
29     }
30     } else
31     {
32         if isdigit(ch) {
33             ReadingDigits = 1;
34             printf(BOLD);
35         }
36         putchar(ch);
37     }
38 }
39 void main()
40 {
41     PrintBoldDigits();
42 }

```

Tanken är att `ReadingDigits` ska vara *sant* när vi är i färd med att läsa in siffror. Från början är variabeln *falsk* ($= 0$, rad 21).

Normalt skriver vi bara ut det tecken som just lästs (rad 35). Det intressanta händer i övergångarna. När vi övergår

- från sifferläsning till bokstavsläsning
- eller från bokstavsläsning till sifferläsning

då ska `<b>` eller `</b>` skrivas ut.

Övergångarna finns där `ReadingDigits` och `isdigit()` inte är överens. När `ReadingDigits` är *sant* men nästa tecken är en icke-siffra (rad 25+26). Eller tvärtom, när `ReadingDigits` är *falsk* och nästa tecken är en siffra (rad 25+31).

Då skrivs `<b>` eller `</b>` ut. Dessutom uppdateras `Readingdigits`.

Hängande `else`

Jag fick oväntat problem i programmet med ett "hängande `else`".

Ibland uppstår en tvetydig situation i användandet av en `if` inuti en annan `if`. Problemet är: hör `else` till det första eller andra `if`?

Exempel. Följande `if`-satser är ganska lika. Men om `x = 1` händer olika saker.

(A)

```

1  if(x > 0) {
2      if(x > 1)
3          printf("Hej!");
4  }
5  else
6      printf("Oj!");
7  /* Inget skrivs ut */

```

(B)

```

1  if(x > 0) {
2      if(x > 1)
3          printf("Hej!");
4      else
5          printf("Oj!");
6  }
7  /* "Oj!" skrivs ut */

```

Skillnaden ligger i var parenteserna ligger -- i vad som omfattas av den första `if`-kroppen. Men om vi tar bort parenteserna, vilket är fullt tillåtet, vad händer då?

```
1  if(x > 0)
2      if(x > 1)
3          printf("Hej!");
4  else
5      printf("Oj!");
```

Svaret är att alternativ (B) gäller. Trots att jag har försökt luras genom att indentera koden enligt alternativ (A).

I C gäller följande: ett hängande `else` kopplas i ihop med det senaste `if`-et.

Problemet kan alltid lösas genom med mjuka parenteser. I mitt program ovan har jag ramat in raderna 26 till 29.

Tydligare version

Den andra varianten av `PrintBoldDigits()` är lite tydligare. I och med att den inte innehåller flera nivåer av `if`-satser.

Vi slipper också problemet med hängande `else` -- det finns inga `else`.

För övrigt fungerar denna version i princip likadant som den första.

```
19 void PrintBoldDigits()
20 {
21     int ReadingDigits = 0;
22     int ch;
23
24     while ((ch = getchar()) != EOF) {
25         if (ReadingDigits && !isdigit(ch)) {
26             ReadingDigits = 0;
27             printf(NO_BOLD);
28         }
29         if (!ReadingDigits && isdigit(ch)) {
30             ReadingDigits = 1;
31             printf(BOLD);
32         }
33         putchar(ch);
34     }
35 }
```

Titta på rad 25 och 29. Det intressanta händer när `isdigit(ch)` och `ReadingDigits` har *olika* värden. Det är då vi ska ändra värde på `ReadingDigits`, och det är då vi ska skriva ut fettningssträngarna.

Kort version -- med förhinder

Den tredje varianten av programmet tar fasta på detta! Att det är när `ReadingDigits != isdigit(ch)` som det intressanta händer.

```
18 int isdigit(int ch)
19 {
20     return '0' <= ch && ch <= '9';
21 }
22
23 void PrintBoldDigits()
24 {
```

```

25     int ReadingDigits = 0;
26     int ch;
27
28     while ((ch = getchar()) != EOF) {
29         if (ReadingDigits != isdigit(ch)) {
30             ReadingDigits = !ReadingDigits;
31             printf(ReadingDigits ? BOLD : NO_BOLD);
32         }
33         putchar(ch);
34     }
35 }

```

Programmeraren var tvungen att lösa ett oväntat problem. Det gick inte att jämföra `ReadingDigits` med värdet från `isdigit(ch)` (rad 29)!

Funktionen `isdigit()` i `ctype.h` utnyttjar nämligen en egenhet hos C när det gäller logiska värden -- att allt är *sant* utom noll: `isdigit()` returnerade flera olika heltal som representation av *sant*!

Det är i och för sig tillåtet, men det saboterade jämförelsen på rad 29. Programmeraren blev tvungen att skriva en egen version av `isdigit()`.

Vilken version är bäst?

Den andra versionen är bäst. För den är tydligast. Och tydlighet *regerar*.

Jag tycker den skulle bli snäppet ännu tydligare om raderna 25--28 bytte plats med raderna 29--32. Då kommer *if*-satserna i samma ordning som de kommer att bli sanna när programmet körs. Möjligen är jag pedant...

Å andra sidan är den tredje versionen ganska kort. Och korthet är *på*. Så version tre får tröstpris.

Övningar

- Modifiera ett av programmen så att det även fetar decimalpunkter -- men bara om decimalpunkten ingår i ett tal!
- Modifiera ett av programmen så att det skippar tomrum, dvs så att det skriver `<b>123 456</b>` i stället för `<b>123</b> <b>456</b>`.

Nästa lektion

